

PersonalAI: A Systematic Comparison of Knowledge Graph Storage and Retrieval Approaches for Personalized LLM agents

MIKHAIL MENSCHIKOV¹, DMITRY EVSEEV¹, Victoria Dochkina², Ruslan Kostoev², Ilia Perepechkin², Petr Anokhin³, Evgeny Burnaev¹ and Nikita Semenov¹

¹Skoltech, Moscow, Russia

²Public joint stock company "Sberbank of Russia", Moscow, Russia

³AIRI, Moscow, Russia

Corresponding author: Mikhail Menschikov (e-mail: m.menschikov@skoltech.ru).

ABSTRACT Personalizing language models that effectively incorporating user interaction history remains a central challenge in development of adaptive AI systems. While large language models (LLMs), combined with Retrieval-Augmented Generation (RAG), have improved factual accuracy, they often lack structured memory and fail to scale in complex, long-term interactions. To address this, we propose a flexible external memory framework based on knowledge graph, which construct and update memory model automatically by LLM itself. Building upon the AriGraph architecture, we introduce a novel hybrid graph design that supports both standard edges and two types of hyper-edges, enabling rich and dynamic semantic and temporal representations. Our framework also supports diverse retrieval mechanisms, including A*, water-circle traversal, beam search and hybrid methods, making it adaptable to different datasets and LLM capacities. We evaluate our system on three benchmarks: TriviaQA, HotpotQA, DiaASQ and demonstrate that different memory and retrieval configurations yield optimal performance depending on the task. Additionally, we extend the DiaASQ benchmark with temporal annotations and internally contradictory statements, showing that our system remains robust and effective in managing temporal dependencies and context-aware reasoning.

INDEX TERMS GraphRAG, Graph Traversal Approaches, Knowledge Graphs Generation, MultiAgency, Question Answering,

I. INTRODUCTION

RECENT advances in large language models (LLMs) have sparked growing interest in personalized AI systems capable in adapting to users based on their interaction history. Central to personalization is the challenge of encoding, storing and retrieving relevant information over long time horizons in a manner that supports efficient reasoning and response generation. While Retrieval-Augmented Generation (RAG) has become a widely used solution, enhancing factual recall by appending retrieved content to prompts, it remains limited by its unstructured nature and weak support for semantic relationships across stored memories.

In this work, we introduce a flexible graph-based memory framework designed to overcome these limitations by enabling structured, customizable representations of long-term memory and supporting advanced reasoning capabilities. Unlike traditional RAG pipelines that rely on dense vector similarity over raw text chunks, our system supports multiple memory formats: nodes, knowledge triples, thesis

statements, episodic traces and dynamically organizes them into a knowledge graph. This structure allows the agent to represent, update and access semantic and temporal relationships with far greater control and interpretability.

Equally important, our framework supports a pluggable retrieval interface with multiple traversal mechanisms, including three variations of A* search, WaterCircle traversal, BeamSearch and hybrid strategies to adapt retrieval behavior to task demands and model capacity. We demonstrate that different memory and retrieval configurations yield optimal performance on different benchmarks and LLM scales, highlighting the versatility of our approach.

We build our system on top of the AriGraph architecture [1], originally developed for LLM agents in interactive text environments such as TextWorld. AriGraph continuously maintains a structured knowledge base by extracting triples from observations, pruning outdated or redundant facts and integrating episodic and semantic vertices into a unified graph. Our framework extends this foundation by allowing

customization of both save (memorization) and search (retrieval) modules, supporting task-specific tuning and component evaluation.

In summary, our main contributions are as follows:

- 1) We present a highly flexible external memory architecture that can be tuned via orthogonal hyperparameters for storage and retrieval.
- 2) We propose and evaluate six retrieval over structured knowledge graph, achieving superior performance across various datasets compared to GraphRAG baselines.
- 3) We enhance the DiaASQ benchmark by incorporating temporal structures into dialogue representations and demonstrate that our framework can leverage such structures to improve temporal reasoning.

This work provides a general and extensible framework for integrating long-term memory and adaptive reasoning into LLM agents, advancing the state of personalized and context-aware language generation.

II. RELATED WORK

In recent years, substantial advancements have been made in open-domain question answering (QA) and the personalization of language models. Techniques leveraging Wikipedia as a broad knowledge source [2] have successfully incorporated large-scale machine reading, integrating document retrieval with textual comprehension. Furthermore, dense representation-based methods for passage retrieval [3] have demonstrated superior efficacy compared to traditional sparse retrieval approaches, such as TF-IDF and BM25, particularly in scenarios with sufficient training data.

The development of dense retrievers has demonstrated significant progress, particularly through integration of contrastive learning with unsupervised settings, which has shown promising results across diverse scenarios and outperformed conventional approaches such as BM25 [4] and Contriever [5]. Concurrently, pre-trained language models incorporating non-parametric memory access have been proposed for knowledge-intensive tasks. A notable example is retrieval-augmented generation (RAG) models, which integrate parametric and non-parametric memory mechanisms to improve question-answering performance [6].

Recent advancements in unsupervised dense retrieval models, such as ART, have demonstrated the ability to achieve state-of-the-art performance while eliminating reliance on labeled training data [7]. In the domain of knowledge graph-based approaches, frameworks such as GraphReader [8] incorporate structured reasoning mechanisms to facilitate knowledge extraction and representation, with a particular emphasis on enhancing long-context reasoning capabilities.

For personalized models, AriGraph [1] introduces a framework that integrates episodic memory and long-term planning using knowledge graphs. Similarly, HippoRAG [9] employs personalized algorithms to improve question-answering (QA) performance by constructing semantic graphs, demonstrating notable advancements over conventional extraction methods.

MemWalker [10] and PEARL [11] address challenges associated with context length, proposing architectures capable to efficiently traversing and consolidating information from large-scale documents.

Additionally, ReadAgent [12] addresses the challenge of processing long-text contexts by structuring content into memory episodes. Meanwhile, KGP [13] proposes Knowledge Graph Prompting, a method that enhances multi-document question answering by constructing knowledge graphs to improve contextual reasoning.

These advancements demonstrate a sustained emphasis on improving language models comprehension, retrieval, and personalization capabilities. Such progress facilitates the development of more sophisticated systems, which leverages knowledge graphs to deliver personalized and contextually enriched interactions.

III. METHODS

A. MEMORY STRUCTURE

In this study, we employ a graph knowledge base as external memory to enhance a large language model's (LLM) question-answering capabilities. Memory model $G = (V_o, E_o, V_t, E_t, V_e, E_e)$ consists of *semantic* (V_o, E_o, V_t, E_t) and *episodic* (V_e, E_e) memory vertices and edges. In turn, semantic vertices and edges are classified into *theses* and *objects*. To capture and structure information from weakly structured sources, specifically, unstructured natural language texts d_i , the construction of that memory is performing by LLM in automatic manner. This memory graph provides a comprehensive representation of the original texts and comprises the following elements (see Figure 1):

- 1) V_o is a set of object vertices. Each object vertex represents atomic concept extracted from corresponding d_i .
- 2) E_o is a set of object edges. Object edge is a tuple (v, rel, u) , where v, u are object vertices and rel is a text-attributed relationship between them, that captures its direct associations. Object edges essentially represent triples integrated in the *semantic memory*.
- 3) V_t is a set of thesis vertices. Thesis vertex encapsulate complete atomic thought, expressed in the corresponding d_i .
- 4) E_t is a set of thesis edges. Thesis edges serves as hyper-edges, linking corresponding set of object vertices, extracted from the same d_i and belonging to the given v_t^i .
- 5) V_e is a set of episodic vertices. Each episodic vertex corresponds to an original text passage ($v_e^i = d_i$) and serve as hyper-edge, linking corresponding/relative vertices.
- 6) E_e is a set of episodic edges. Each episodic edge $e_e^i = (v_s^i, v_e^i)$ connects all semantic vertices V_s^i , extracted from d_i , with each other by the corresponding episodic vertex v_e^i .

B. MEMORY CONSTRUCTION

By the terminology defined above, the process of constructing a memory graph from weakly structured sources can be

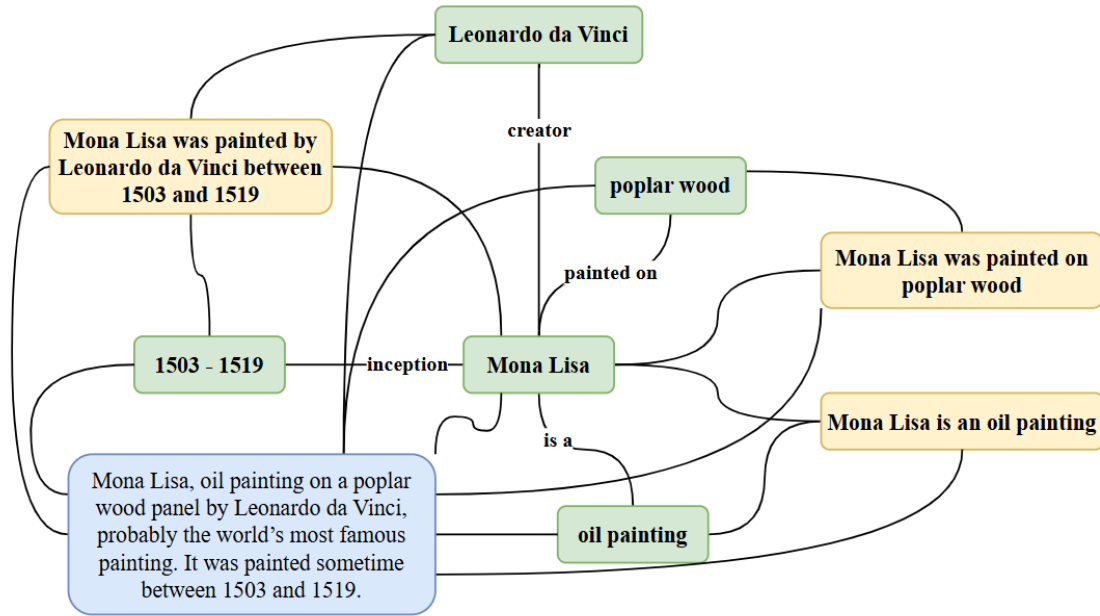


FIGURE 1. Example of a graph fragment, constructed from natural language text using our method, with object (green), thesis (yellow) and episodic (blue) vertices

decomposed into three key steps: (1) formulating vertices and their edges, (2) generating hyper-edges using an LLM, and (3) parsing the LLM's output to store extracted information in a structured format (e.g., as subject-relation-object triples). An LLM prompts for extracting semantic memories from textual sources is provided in Appendix A. Full memory construction pipeline (Memorize pipeline) is demonstrated on Figure 2.

The identification and retrieval of outdated information within the memory are performed through the following procedure. First, the vertices present in the newly extracted triples are compared against existing vertices in the memory graph to detect matches. Upon identifying matching vertices, these serve as the initial set for a breadth-first search (BFS), which traverses all associated standard- and hyper-edges. Subsequently, a specialized prompt is employed to instruct LLM to update retrieved knowledge with newly extracted data (see Appendix B for corresponding LLM prompts). If any triple are successfully updated, the corresponding outdated instance are removed from memory model.

C. INFORMATION SEARCH IN MEMORY

Information search pipeline (QA pipeline) for memory model is shown in Figure 3.

As illustrated in Figure 3, pipeline comprises four primary stages, with the third stage further subdivided into two sub-stages. QA pipeline begins by accepting a natural language question as input, which is then processed by the **Entities Extractor** module to extract key entities from it. In the second stage, these entities are passed to the **Entities to Vertices Matcher** module, where they are aligned with corresponding/relevant vertices in the memory graph.

The third stage involves two sequential operations: retrieval

and filtering of graph triples based on semantic similarity to the input question. First, the **Memory Graph Triples Retriever** module initiates a graph traversal algorithm, using the matched entities from the second stage as starting vertices to retrieve a set of candidate triples. Subsequently, the **Triples Filter** module ranks these triples by computing their semantic similarity to the question via vector embeddings, retaining only the top N (a predefined hyperparameter) most relevant items.

Finally, in the fourth stage, the **Conditional Answer Generator** module synthesizes a natural language answer, conditioned on a retrieved/filtered triples. The output of QA pipeline is the generated answer in a string format. Used LLM prompts for described modules is provided in Appendix C.

This pipeline architecture is designed based on three key considerations. First, to achieve an accurate initial approximation of the relevant subgraph, it is essential to align the key entities in the input question with semantically similar vertices in the existing memory graph. The reasoning behind this is that the information necessary for generating a correct response is typically localized within the subgraph containing these key entities. Second, the triples extracted from the knowledge graph exhibit only weak conditioning on the input question, limiting their direct applicability. Third, large language models (LLMs) are constrained by a fixed maximum sequence length for processing inputs in a single inference step, necessitating efficient retrieval and subgraph selection strategies.

D. RETRIEVAL ALGORITHMS

The primary role of the constructed memory graph in our proposed system is to enable the accurate retrieval of infor-

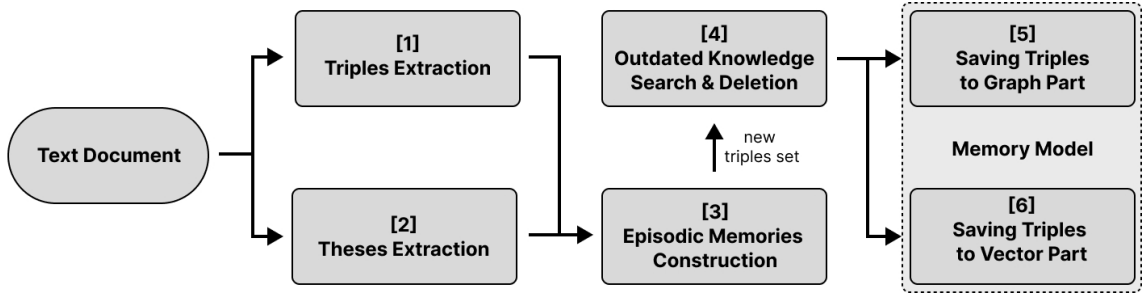


FIGURE 2. High level architecture of proposed Memorize pipeline for LLM-based triples extraction from unstructured texts on natural language and memory construction

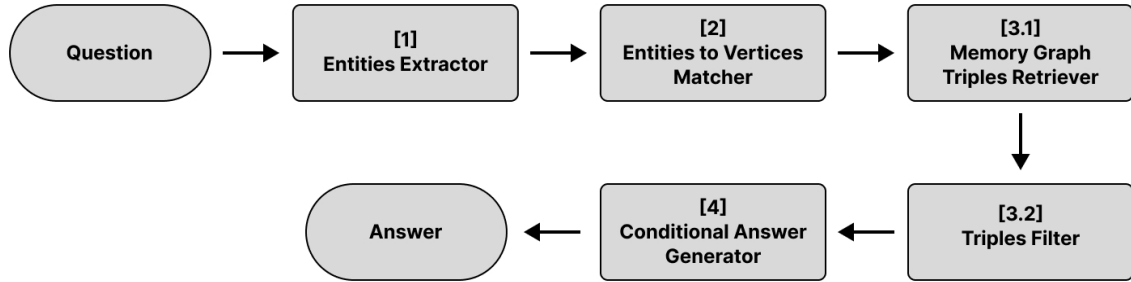


FIGURE 3. High level architecture of proposed QA pipeline for generating answers to the questions based on constructed memory graph

mation required for responding to specific user questions. This process necessitates a careful optimization between two key criteria: relevance and completeness. Relevance ensures that all retrieved information is directly pertinent to the question, whereas completeness guarantees the inclusion of all necessary contextual data, even if some retrieved elements may be extraneous. Consequently, striking an optimal balance between these factors is essential for achieving efficient and effective knowledge extraction.

To accomplish this, we design and implement advanced retrieval algorithms capable of dynamically balancing the trade-off between completeness and relevance through configurable parameter settings. These algorithms constitute a critical component of the question-answering pipeline, systematically traversing the memory graph to aggregate information essential for generating precise and contextually appropriate responses.

A*. The A* algorithm is a widely used method for graph traversal, particularly valued for its ability to efficiently identify shortest paths between vertices in a graph. In the context of our question-answering pipeline, this algorithm extracts and retains triples encountered along these shortest paths while eliminating duplicates based on a content. We hypothesize that triples, obtained through this process, contains necessary information for generating accurate responses. For traversal, we treat our memory graph as unweighted and undirected, using a constant distance metric between adjacent vertices. To optimize pathfinding efficiency, we evaluate three distinct heuristics for the h -metric:

- 1) **Inner Product (IP):** This heuristic computes dot product of embeddings between current and target vertices.

- 2) **Weighted Shortest Path:** This approach scales inner product metric by the length of shortest path, determined via Breadth-First Search (BFS).
- 3) **Averaged Weighted Shortest Path:** This heuristic calculates average of inner product distances between adjacent vertices along the path from start vertex to current vertex, as well as the direct path from current vertex to target vertex, and further weights this value by BFS-derived shortest path length.

WaterCircles. This graph-based extraction method employs a breadth-first search (BFS) algorithm to retrieve relevant knowledge. Query entities are first mapped to their corresponding/relevant vertices in the memory graph. Algorithm begins with exploring adjacent vertices of initial vertices and iteratively expands to neighboring vertices in subsequent steps, constructing radiate pathways. When pathways originating from different starting vertices intersect, triples, formed at these intersections, are aggregated into a primary list, while all traversed triples are compiled into a secondary list. Algorithm ultimately returns a subset of triples, selecting N from the primary list and K from the secondary list, where N and K are configurable hyperparameters.

When the memory graph incorporates not only direct object relations (*object* triples), but also associations between objects and text fragments (referred to *thesis* and *episodic* triples), a modified breadth-first search (BFS) algorithm is employed as follows:

- 1) **Traversal Initialization:** The search begins at vertices, that matches entities from the input question.
- 2) **Text Fragment Analysis:** During traversal, identified text fragments are examined for occurrences of other

question entities, distinct from the entity that originated the path to the given fragment. For each fragment, the number of detected entities, denoted as $N_{intersections}$, is computed.

- 3) **Ranking Triplets:** The list of *thesis* and *episodic* triplets is then sorted in descending order based on $N_{intersections}$.

This triplet extraction strategy aims to enhance relevance and accuracy in retrieving information from the memory graph, thereby bolstering the effectiveness of AI-driven question-answering systems.

BeamSearch. Given a starting vertex, algorithm constructs N (a hyperparameter) semantically relevant paths in response to the input question. This approach is inspired by beam search, a token generation strategy commonly employed in large language model (LLM) inference. The resulting paths are consolidated into a single list, with duplicate triples removal based on their string content. Traversal process is governed by the following hyperparameters:

- 1) **Max depth:** The maximum allowable depth for path construction.
- 2) **Max paths:** The maximum number of paths to generate.
- 3) **Same Path Intersection by Vertex:** If enabled, a path may revisit a vertex; otherwise, vertex revisitation is prohibited.
- 4) **Diff Paths Intersection by vertex:** If enabled, distinct paths may share vertices; otherwise, vertex sharing is disallowed.
- 5) **Diff Paths Intersection by Edge:** If enabled, distinct paths may share edges; otherwise, edge sharing is forbidden.
- 6) **Final sorting mode:** Determines the method for filtering the final set of paths. The search yields two path subsets: (1) *ended_paths* – path terminated before reaching the depth limit and (2) *continuous_paths* – paths that reached the depth limit. Each path is assigned a relevance score. The selection process varies based on the chosen mode. If *ended_first*–value is specified, then: *ended*–paths will be sorted in descending order by relevance and the first *max_paths*–paths will be selected. If there are fewer *ended*–paths than *max_paths*–value, then *continuous*–paths will be sorted by relevance and the first N missing paths will be selected from them. If *continuous_first*–value is specified, then paths will be selected in the same way as *ended_first*–value, only first sorting/selection by *continuous*–paths, and then by *ended*–paths. If *mixed*–value is specified, then *ended*– and *continuous*–paths will be combined into one list, sorted in descending order by relevance, and the first N *max_paths*–paths will be selected from the resulting list.

Mixed Algorithm. This algorithm integrates A*, WaterCircles and BeamSearch strategies to enhance extraction efficacy. By combining these approaches, we ensures that triples

not captured by one method (e.g., WaterCircles) may still be retrieved by another (e.g., BeamSearch), thereby improving overall recall. The final set of triples, which is subsequently passed to the LLM for answer generation, constitutes the union of outputs derived from A*, WaterCircles and BeamSearch algorithms.

By evaluating these diverse algorithms, this study highlights advancements in extracting pertinent information from knowledge graphs, thereby supporting the robust architectural framework required for personalizing responses in large language model (LLM) agents.

IV. EXPERIMENT SET-UP

A. DATASETS

To evaluate proposed retrieval algorithms, we conducted experiments across three distinct benchmarks: DiaASQ [14], HotpotQA and TriviaQA. This selection was designed to evaluate our framework across varying domains, structural complexities and reasoning requirements.

The primary evaluation dataset is DiaASQ, which consists of user dialogues from a Chinese forum focused on mobile device characteristics. A key feature of this dataset is the inclusion of structured "true statements" that encapsulate the core semantic content of each dialogue. We procedurally generated evaluation questions from these statements to ensure precise assessment. To further evaluate framework's capability in handling temporal dynamics and contradictory information, critical for personalized agents, we extended DiaASQ with explicit temporal annotations and internally contradictory statements.

To ensure broad applicability and mitigate potential dataset bias and limited domain diversity, we supplemented DiaASQ with two widely-used, general-domain QA benchmarks:

- 1) HotpotQA: Selected for its requirement of multi-hop reasoning across multiple documents.
- 2) TriviaQA: Chosen for its factoid-style, open-domain questions that test broad knowledge retrieval.

These datasets provide complementary challenges, moving beyond a single domain (mobile devices) to perform evaluation on general world knowledge and complex reasoning.

Considering the computational and engineering complexity of constructing and traversing large memory graphs, we created manageable yet representative subsets from the original datasets. This step was necessary to enable the iterative experimentation required for tuning multiple retrieval algorithms and LLM configurations within practical resource constraints. The preprocessing involved filtering contexts by length and, for TriviaQA, segmenting long documents into coherent chunks. The resulting subsets used for knowledge graph construction and QA pipeline evaluation are summarized in Table 1. Detailed preprocessing steps and dataset statistics are provided in Appendix D.

Using these benchmarks, memorization and question-answering functionality of our PersonalAI framework is systematically compared against existing Retrieval-Augmented

Dataset	#qa-pairs	#contexts
DiaASQ	4800	3483
HotpotQA	2000	3933
TriviaQA	500	4925

TABLE 1. Characteristics of prepared datasets for QA pipeline evaluation

Generation (RAG) and GraphRAG baselines, as detailed in subsequent sections.

B. MODELS

For memory graph construction (Memorize pipeline) and information retrieval (QA pipeline) we evaluate a series of 7B/8B and large-scale ($\gg 14$ B) language models to assess their performance in handling these tasks. The selected models include Qwen2.5 7B, DeepSeek R1 7B, Llama3.1 8B, GPT4o-mini, and DeepSeek V3. To generate vector representations (embeddings) of natural language text data, we employ a multilingual E5-large model¹.

C. GRAPH-TRAVERSAL ALGORITHMS

Graph-traversal algorithms evaluation is conducted for A*, WaterCircles (WC) and BeamSearch (BS) as well as their combinations: "WC + BS", "A* + BS", "A* + WC". The values of hyperparameters for base algorithms were fixed (see Appendix E).

D. GRAPH-TRAVERSAL RESTRICTIONS

Additionally, we systematically varied the values of hyperparameters governing the traversal constraints applied to the graph during algorithm execution. These constraints determine which vertex types are excluded from traversal: "E" prohibits traversal of episodic vertices, "T" prohibits traversal of thesis vertices, and "O" prohibits traversal of object vertices. The "all" key word means that restrictions about graph vertices traversal is not applied.

E. SUMMARY OF EXPERIMENT CONFIGURATIONS

Each QA configuration was evaluated on 100 question-answer pairs from DiaASQ, HotpotQA, and TriviaQA datasets. The same LLM was used both for generating responses within a given QA configuration and for constructing the corresponding memory graph to execute the QA pipeline. Consequently, for each fixed dataset/model pairing, 22 distinct QA configurations were derived. In total, 308 QA configurations were evaluated (the QA pipeline was not executed on TriviaQA/GPT4o-mini configurations due to resource constraints).

F. MEMORY CONSTRUCTION SETTING

Our implementation of memory consists of two main parts: graph part and vector part. The graph part stores textual

representations of object, thesis and episodic vertices, their properties and relationships (edges) between them. Neo4j is used as the graph database for this part of the system. The vector part of memory stores vector representations (embeddings) of elements from the graph part to measure semantic similarity of texts for QA pipeline execution. Milvus is used as a vector database for this part of the system. Our memory model also implements a caching mechanism for storing intermediate results of QA pipeline components to reduce overall time spent to process incoming questions. This component utilizes two non-relational databases: Redis and MongoDB. During our experiments cache was enabled. All used databases were hosted and run on a single machine in personal docker containers. Medium size LLM (7B/8B) were used from local hosted Ollama docker container. LLM inference during memory construction and QA pipeline processing performed on a single NVIDIA TITAN RTX 24GB.

To evaluate QA pipeline we construct 14 memory graphs based on described datasets and LLMs above. Important to note, that we disable stage there we searching and deleting outdated knowledge in memory because of the experiments scope. An average speed of adding text fragments per minute to the memory for a given database configuration is ~ 1.35 (with an average length of a processing texts 550-650 characters). Detailed characteristics of constructed memory graphs can be observed in Appendix G.

V. EVALUATION

Traditional statistical evaluation metrics such as *BLEU* [15], *ROUGE* [16], and *Meteor Universal* [17] struggle to distinguish syntactically similar but semantically distinct texts. While semantic methods like *BERTScore* [18] were introduced to address these limitations, our experiments reveal that *BERTScore* lacks sufficient differentiability, often failing to capture nuanced distinctions between correct and incorrect answers. Therefore, we adopt the *LLM as a judge* [19] framework and choose **Qwen2.5 7B**. The judge evaluates QA pairs using a structured prompt containing the question, ground truth and model answer. It labels 1 for correct answers and 0 for incorrect ones, and we use accuracy as our main metric. Corresponding LLM-prompts and details are provided in Appendix F.

Also, for comparison of proposed QA pipeline with existing RAG and GraphRAG methods the *ExactMatch* metric is calculated with "ignore_case" and "ignore_punctuation" hyperparameters set to True.

VI. EXPERIMENTS AND RESULTS

Based on experimental results, a comparative table, summarizing best-performing QA configurations by LLM-as-a-Judge metric, was compiled (see Table 2).

As shown in Table 2, Qwen2.5 achieved best performance (0.27) among 7B models. Among all evaluated configurations, highest overall effectiveness (0.77) was reached by setups incorporating GPT-4o-mini. Notably, top-performing 7B configurations predominantly relied on BeamSearch, es-

¹<https://huggingface.co/intfloat/multilingual-e5-small>

LLM / Dataset	Qwen2.5 7B	DeepSeek R1 7B	Llama3.1 8B	GPT4o-mini	DeepSeek V3
DiaASQ	0.22 / BS / all	0.12 / AS / E	0.19 / BS / E	<u>0.5</u> / BS + WC / E	0.47 / BS + WC / O
HotpotQA	0.24 / BS / O	0.19 / BS / O	0.47 / BS / O	<u>0.77</u> / BS + WC / all	0.76 / BS + WC / T
TriviaQA	0.34 / BS / E	0.27 / AS / E	0.66 / BS + AS / E	–	<u>0.87</u> / BS + WC / all
Mean:	0.27	0.19	0.44	0.77	0.70

TABLE 2. Best QA configurations ranked by LLM-as-a-Judge metric across all experiments. Corresponding cells contains LLM-as-a-Judge score, used retrieval algorithm and the type of restriction applied to the graph during traversal. Shortcuts for retrieval algorithms: BA – BeamSearch; AS – A*; BS + AS – hybrid of BeamSearch and A*; BS + WC – hybrid of BeamSearch and WaterCircles. Shortcuts for graph restrictions: all – no restrictions applied; E – episodic vertices excluded from traversal; T – thesis vertices excluded; O – object vertices excluded.

pecially under constraint that traversal restricted through episodic vertices. In contrast, best DeepSeek V3 configurations frequently adopted a hybrid strategy combining BeamSearch and WaterCircles. Across high-performing configurations more broadly, BeamSearch consistently appeared as a key component within the retrieval pipeline.

To evaluate the effect of imposed constraints on the quality of the QA pipeline, we construct two distinct distributions of values, each corresponding to a specific model/dataset pair and retrieval algorithm. These distributions then were averaged across datasets and models for configurations exhibiting lowest and highest LLM-as-a-Judge values, as detailed in Table 3.

As demonstrated in Table 3, for 7B/8B models, majority ($\approx 74\%$) of configurations yielding the lowest response quality impose restrictions on traversing thesis-type vertices. Conversely, a significant proportion of high-quality configurations restrict traversal of episodic and object vertices ($\approx 44\%$ and $\approx 34\%$ respectively). This suggests that thesis-type memories contain critical information for generating relevant responses, whereas the inclusion of episodic and object memories introduces noise into the context, thereby degrading output quality. For larger-scale models, the trend differs: 53% of low-quality configurations restrict traversal of object vertices, while 73% of high-quality configurations restrict traversal of thesis-type vertices. This implies that larger models exhibit greater robustness in handling conditional generation from lengthy or noisy episodic memories, rendering thesis-based information redundant.

Additionally, a comparative analysis was conducted, as presented in Table 4, to evaluate the robustness of various graph traversal algorithms. The results indicate that configurations employing 8B models in conjunction with a combined search strategy (A* and WaterCircles) demonstrate high stability, with performance degradation remaining within 4% across varying traversal constraints. In contrast, the BeamSearch algorithm exhibits high sensitivity to these constraints: suboptimal parameterization results in substantial performance loss, with LLM-as-a-Judge varying by as much as 24% between optimal and non-optimal settings. However, in the

case of larger-scale models, the combination of BeamSearch and WaterCircles algorithms yields more consistent performance, suggesting improved robustness at higher model capacities.

A critical component of the implemented QA pipeline is the "NoAnswer" mechanism. This mechanism incorporates a directive in the LLM prompt, instructing the model to output a predefined symbol if provided context lacks sufficient information to generate a valid response. Table 5 summarizes the frequency of "NoAnswer" outputs across different model configurations, retrieval algorithms, and graph traversal constraints.

As demonstrated in Table 5, the analysis reveals distinct patterns in the occurrence of "NoAnswer" responses across different model configurations. For 7B models, lowest frequency of "NoAnswer" responses is observed when employing a combined A* and WaterCircles algorithm with restricted traversal of thesis vertices. In contrast, 8B models exhibit minimal "NoAnswer" instances when utilizing the BeamSearch algorithm alongside a prohibition on episodic vertices traversal. For larger-scale models, the optimal performance, measured by the fewest "NoAnswer" responses, is achieved through a combined BeamSearch and WaterCircles approach without graph traversal constraints. These findings suggest that the aforementioned algorithms are more effective at extracting relevant information compared to alternative methods under the specified conditions.

It is also important to note the required time to process one user question by QA pipeline with a given LLM and retrieval algorithm: see Table 6.

Table 6 shows that QA pipeline with WaterCircles retriever are significantly faster. This is because that WaterCircles does not need to use vector part of memory graph with vector representations of its elements to perform traversal. Conversely, QA pipeline with BeamSearch algorithm turned out to be slower than QA pipeline with A* algorithm. This is because A* requires embeddings of memory graph elements to construct and traverse only one path on the graph, while BeamSearch requires to construct and monitor N paths/candidates to select optimal traversal. The obtained

Vertex types restrictions	7B		8B		14B+	
	worse configs	best configs	worse configs	best configs	worse configs	best configs
E	3%	<u>44%</u>	9%	<u>45%</u>	27%	7%
T	<u>84%</u>	12%	<u>64%</u>	31%	20%	<u>73%</u>
O	13%	<u>44%</u>	27%	25%	<u>53%</u>	20%

TABLE 3. Impact of various constraints, imposed during memory graph traversal, on the quality of QA pipeline: "worse configs" – distribution for low quality configurations; "best configs" – distribution for high quality configurations

Retrieval algorithm	7B			8B			14B+		
	worse configs (w restr)	best configs (w restr)	other configs (w/o restr)	worse configs (w restr)	best configs (w restr)	other configs (w/o restr)	worse configs (w restr)	best configs (w restr)	other configs (w/o restr)
WC	–	–	0.09	–	–	0.34	–	–	0.55
AS	<u>0.1</u>	0.175	<u>0.14</u>	0.29	0.36	0.41	0.23	0.36	0.33
BS	0.025	<u>0.18</u>	0.06	0.26	<u>0.5</u>	0.36	0.48	0.6	0.65
WC+BS	0.033	0.095	0.02	0.30	0.39	0.32	<u>0.62</u>	<u>0.7</u>	<u>0.68</u>
BS+AS	0.02	0.175	0.01	0.25	0.48	0.36	0.48	0.64	0.66
AS+WC	0.055	0.115	0.07	<u>0.33</u>	0.37	<u>0.42</u>	0.57	0.6	0.6

TABLE 4. Stability of proposed retrieval algorithms when various restrictions are imposed: "worse-configs (w restr)" – low quality configurations and imposed restrictions; "best-configs (w restr)" – high quality configurations and imposed restrictions; "other-configs (w/o restr)" – configurations without restrictions on graph traversal

Retrieval algorithm / LLM size	WC	AS	BS	WC+BS	BS+AS	AS+WC	Vertex types restrictions / LLM size	all	E	T	O
7B	31%	44%	26%	29%	31%	<u>25%</u>		33%	27%	<u>26%</u>	36%
8B	51%	49%	<u>43%</u>	51%	73%	49%		51%	<u>40%</u>	51%	46%
14B+	25%	62%	27%	<u>16%</u>	26%	21%		<u>26%</u>	32%	27%	35%

TABLE 5. The influence of selected retrieval algorithms and imposed search restrictions on the percentage of generated "NoAnswer" stubs

LLM / Retrieval algorithm	QA pipeline Latency (minutes)					Mean:
	Qwen2.5 7B	DeepSeek R1 7B	Llama3.1 8B	GPT4o mini	DeepSeek V3	
WC	0.14	0.34	0.46	0.22	0.33	0.30
AS	2.24	4.68	3.51	-	2.53	3.24
BS	5.08	7.86	5.00	8.70	6.32	6.59

TABLE 6. QA pipeline Latency (in minutes), depending on LLM and retrieval algorithm with enabled caching mechanism

Latency of the traversal algorithms is also highly dependent on the chosen vector database then configuring vector part of memory. In our experiments Milvus was used. After completing our main experiments, we evaluated read/write

performance of five databases (Milvus, OpenSearch, Weaviate, ElasticSearch, and Qdrant) and found the following: (1) Qdrant was the fastest database, and (2) Qdrant in six times faster than Milvus. Therefore, we recommend speci-

fying/using Qdrant when configuring our memory graph to reduce average required time to process single question by our proposed QA pipeline.

In summary, our framework demonstrates that the accuracy can be increased by configuring the memory graph ontology and retrieval methods according to the available LLM and selected QA task. A comparative analysis of existing RAG and GraphRAG methods against our proposed approach, conducted on the TriviaQA and HotpotQA datasets, is provided in Appendix H.

VII. CONCLUSION

This work introduces a flexible and extensible framework for integrating structured memory into language model agents based on a knowledge graph. By extending AriGraph architecture with support of object, episodic, thesis vertices and hyper-edges we enable rich temporal and semantic representations that go beyond traditional RAG pipelines. Our system supports multiple retrieval algorithms, including A*, WaterCircles, BeamSearch and hybrid combinations that can be dynamically adapted to the model's scale and task requirements. Through extensive evaluation on three benchmarks (DiaASQ, HotpotQA, and TriviaQA), we demonstrate that performance varies systematically with the choice of retrieval strategy and graph traversal constraints. For smaller-scale models (7B–8B), configurations that restrict episodic or object vertices and rely on BeamSearch yield the highest accuracy, while for larger models, hybrid methods combining BeamSearch and WaterCircles offer improved stability and robustness. Importantly, we show that thesis vertices often encode critical information, and excluding them typically degrades performance, especially in 7B models. Compared to existing RAG and GraphRAG methods, our approach demonstrates competitive or superior performance, particularly in handling temporally complex and contradictory information. Ablation studies further reveal that hybrid traversal strategies reduce sensitivity to graph constraints and lower the frequency of invalid responses. Overall, our system provides a principled architecture for long-term, structured memory in LLM agents, enabling personalized, context-aware reasoning at scale. It lays the groundwork for future extensions, involving: temporal filtering, edge-type prioritization and more fine-grained memory control.

VIII. FUTURE WORK

In future work, we propose to enhance the temporal dynamics of our memory graph by introducing a "memory time" parameter, which will enable fine-grained filtering of triples based on temporal proximity and edge types. This modification will allow the system to selectively prioritize temporally proximate data or emphasize specific relationship categories, thereby improving the precision of personalized responses. Also, one possible future direction is to explore erasure-coded and locally recoverable layouts for sharding graph and vector indices across nodes, inspired by information-theoretic distributed storage, enabling fast repair and continued operation

under partial server unavailability [20]. In addition, we will investigate private and verifiable retrieval protocols (PIR with result verification) so an agent can query remote memory without revealing the user's intent and can detect incorrect or malicious responses [21]. These mechanisms aim to make long-term personalized memory robust, secure, and auditable at scale.

REFERENCES

- [1] Petr Anokhin, Nikita Semenov, Artyom Sorokin, Dmitry Evseev, Mikhail Burtsev, and Evgeny Burnaev. Arigraph: Learning knowledge graph world models with episodic memory for llm agents, 2024.
- [2] Danqi Chen, Adam Fisch, Jason Weston, and Antoine Bordes. Reading Wikipedia to answer open-domain questions. In Regina Barzilay and Min-Yen Kan, editors, *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1870–1879, Vancouver, Canada, July 2017. Association for Computational Linguistics.
- [3] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu, editors, *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, Online, November 2020. Association for Computational Linguistics.
- [4] Stephen Robertson, Hugo Zaragoza, et al. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends® in Information Retrieval*, 3(4):333–389, 2009.
- [5] Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. Unsupervised dense information retrieval with contrastive learning, 2021.
- [6] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2021.
- [7] Devendra Singh Sachan, Mike Lewis, Dani Yogatama, Luke Zettlemoyer, Joelle Pineau, and Manzil Zaheer. Questions are all you need to train a dense passage retriever. *Transactions of the Association for Computational Linguistics*, 11:600–616, 2023.
- [8] Shilong Li, Yancheng He, Hangyu Guo, Xingyuan Bu, Ge Bai, Jie Liu, Jiaheng Liu, Xingwei Qu, Yangguang Li, Wanli Ouyang, Wenbo Su, and Bo Zheng. Graphreader: Building graph-based agent to enhance long-context abilities of large language models, 2024.
- [9] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. Hipporag: Neurobiologically inspired long-term memory for large language models, 2024.
- [10] Howard Chen, Ramakanth Pasunuru, Jason Weston, and Asli Celikyilmaz. Walking down the memory maze: Beyond context limit through interactive reading, 2023.
- [11] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. Raptor: Recursive abstractive processing for tree-organized retrieval, 2024.
- [12] Kuang-Huei Lee, Xinyun Chen, Hiroki Furuta, John Canny, and Ian Fischer. A human-inspired reading agent with gist memory of very long contexts, 2024.
- [13] Yu Wang, Nedim Lipka, Ryan A. Rossi, Alexa Siu, Ruiyi Zhang, and Tyler Derr. Knowledge graph prompting for multi-document question answering, 2023.
- [14] Bobo Li, Hao Fei, Fei Li, Yuhang Wu, Jinsong Zhang, Shengqiong Wu, Jingye Li, Yijiang Liu, Lizi Liao, Tat-Seng Chua, and Donghong Ji. Diaasq : A benchmark of conversational aspect-based sentiment quadruple analysis, 2023.
- [15] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*, pages 311–318, 2002.
- [16] Chin-Yew Lin. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*, pages 74–81, 2004.
- [17] Michael Denkowski and Alon Lavie. Meteor universal: Language specific translation evaluation for any target language. In *Proceedings of the ninth workshop on statistical machine translation*, pages 376–380, 2014.

- [18] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. Bertscore: Evaluating text generation with bert. *ArXiv*, abs/1904.09675, 2019.
- [19] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems*, 36:46595–46623, 2023.
- [20] Stanislav Kruglik and Alexey Frolov. An information-theoretic approach for reliable distributed storage systems. *Journal of Communications Technology and Electronics*, 65:1505–1516, 12 2020.
- [21] Stanislav Kruglik, Son Dau, Han Mao Kiah, and Huaxiong Wang. Two-server private information retrieval with optimized download rate and result verification, 01 2023.
- [22] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. Realm: retrieval-augmented language model pre-training. In *Proceedings of the 37th International Conference on Machine Learning*, ICML'20. JMLR.org, 2020.
- [23] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, NIPS '20, Red Hook, NY, USA, 2020. Curran Associates Inc.
- [24] Omar Khattab, Christopher Potts, and Matei Zaharia. Relevance-guided supervision for OpenQA with ColBERT. *Transactions of the Association for Computational Linguistics*, 9:929–944, 2021.
- [25] Gautier Izacard and Edouard Grave. Leveraging passage retrieval with generative models for open domain question answering. In Paola Merlo, Jorg Tiedemann, and Reut Tsarfaty, editors, *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 874–880, Online, April 2021. Association for Computational Linguistics.
- [26] Devendra Singh Sachan, Siva Reddy, William L. Hamilton, Chris Dyer, and Dani Yogatama. End-to-end training of multi-document reader and retriever for open-domain question answering. In A. Beygelzimer, Y. Dauphin, P. Liang, and J. Wortman Vaughan, editors, *Advances in Neural Information Processing Systems*, 2021.
- [27] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego de Las Casas, Aurelia Guy, Jacob Menick, Roman Ring, T. W. Hennigan, Saffron Huang, Lorenzo Maggione, Chris Jones, Albin Cassirer, Andy Brock, Michela Paganini, Geoffrey Irving, Oriol Vinyals, Simon Osindero, Karen Simonyan, Jack W. Rae, Erich Elsen, and L. Sifre. Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning*, 2021.
- [28] Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. Atlas: few-shot learning with retrieval augmented language models. *J. Mach. Learn. Res.*, 24(1), January 2023.
- [29] Weijia Shi, Sewon Min, Michihiro Yasunaga, Minjoon Seo, Richard James, Mike Lewis, Luke Zettlemoyer, and Wen-tau Yih. REPLUG: Retrieval-augmented black-box language models. In Kevin Duh, Helena Gomez, and Steven Bethard, editors, *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 8371–8384, Mexico City, Mexico, June 2024. Association for Computational Linguistics.
- [30] Jiashuo Sun, Chengjin Xu, Luminyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Heung-Yeung Shum, and Jian Guo. Think-on-graph: Deep and responsible reasoning of large language model with knowledge graph, 2023.
- [31] Linhao Luo, Yuan-Fang Li, Gholamreza Haffari, and Shirui Pan. Reasoning on graphs: Faithful and interpretable large language model reasoning. In *International Conference on Learning Representations*, 2024.
- [32] Jiaxiang Liu, Tong Zhou, Yubo Chen, Kang Liu, and Jun Zhao. Enhancing large language models with pseudo- and multisource- knowledge graphs for open-ended question answering, 2025.
- [33] Yuntong Hu, Zhihan Lei, Zheng Zhang, Bo Pan, Chen Ling, and Liang Zhao. Grag: Graph retrieval-augmented generation. *arXiv preprint arXiv:2405.16506*, 2024.
- [34] Costas Mavromatis and George Karypis. GNN-RAG: Graph neural retrieval for efficient large language model reasoning on knowledge graphs. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Findings of the Association for Computational Linguistics: ACL 2025*, pages 16682–16699, Vienna, Austria, July 2025. Association for Computational Linguistics.
- [35] Shengjie Ma, Chengjin Xu, Xuhui Jiang, Muzhi Li, Huaren Qu, Cehao Yang, Jiaxin Mao, and Jian Guo. Think-on-graph 2.0: Deep and faithful large language model reasoning with knowledge-guided retrieval augmented generation. In *International Conference on Learning Representations*, 2024.
- [36] Jie Ma, Zhitao Gao, Qi Chai, Wangchun Sun, Pinghui Wang, Hongbin Pei, Jing Tao, Lingyun Song, Jun Liu, Chen Zhang, et al. Debate on graph: a flexible and reliable reasoning framework for large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 24768–24776, 2025.
- [37] Linhao Luo, Zicheng Zhao, Chen Gong, Gholamreza Haffari, and Shirui Pan. Graph-constrained reasoning: Faithful reasoning on knowledge graphs with large language models. In *Forty-second International Conference on Machine Learning*, 2025.
- [38] Lei Sun, Xinchun Wang, and Youdi Li. Pyramid-driven alignment: Pyramid principle guided integration of large language models and knowledge graphs. *ArXiv*, abs/2410.12298, 2024.



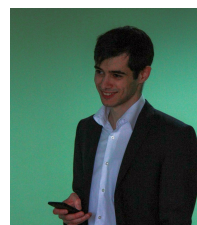
M. MENSCHIKOV received the B.Sc. degree in Software Engineering from Petrozavodsk State University, in 2023, then received M.Sc. degree in Machine Learning Engineering from ITMO University, in 2025. He is currently a Software Engineer at Skoltech AI Center, where he contributed in project for developing working memory for LLM-agents based on knowledge graph. His research interests include generative modeling, GraphRAG, LLM-based knowledge graph reasoning, LLM-base knowledge graph construction, multi-agent and dialog systems.



D. EVSEEV completed master's degree and then postgraduate study at the Moscow Institute of Physics and Technology (The Phystech School of Applied Mathematics and Computer Science), then received the PhD degree. I am currently a Senior Research Engineer at Skoltech.



V. DOCHKINA — Director of the AI and Data Center, Strategy and Development Block, Sber. Leads Block's strategic AI implementation and digital transformation initiatives. Education: Bachelor's and Master's degrees with honors from the Moscow Institute of Physics and Technology (MIPT); Master's degree from Skoltech, recipient of the 2021 Best Thesis Award; ongoing PhD at MIPT focused on multiagent AI systems and foundation model architectures. Expertise: Development and deployment of enterprise-scale AI solutions; AI governance frameworks; Agentic AI. Research interests: Foundation models; multimodal expansion; agentic LLM capability development; scaling AI agents for process automation; autonomous AI systems; mixture-of-experts architectures; coordination frameworks for enterprise-wide autonomization.



R. KOSTOEV got M.Sc. degree in applied mathematics and computer science from Lomonosov Moscow State University, and built an impressive career spanning technology, innovation, and leadership roles. His professional journey includes experience at major companies such as Philips and Google, where he contributed to significant projects and initiatives.



I. PEREPECHKIN got M.Sc. degree in Applied Mathematics and Physics from Moscow Institute of Physics and Technology in 2017. He has experience developing enterprise-level AI solutions. He is currently a team lead data scientist at Sberbank, developing multi-agent systems.



P. ANOKHIN received his M.Sc. degree in Physiology from Lomonosov Moscow State University in 2013 and his Ph.D. in Physiology and Biochemistry from the Russian Academy of Sciences in 2017. His early research focused on studying the dopamine system in animal models of addiction and reinforcement learning. In 2021, he joined AIRI, where he now leads a team researching large language model (LLM) agents, reasoning models, and memory architectures for intelligent agents.



E. BURNAEV received the M.Sc. degree in applied physics and mathematics from Moscow Institute of Physics and Technology, in 2006, the Ph.D. degree in foundations of computer science from the Institute for Information Transmission Problem RAS, in 2008, and the Dr.Sci. degree in mathematical modeling and numerical methods from Moscow Institute of Physics and Technology, in 2022. He is currently the Director of the AI Center, Skolkovo Institute of Science and Technology, and

a Full Professor. His research interests include generative modeling, manifold learning, deep learning for 3D data analysis, multi-agent systems, and industrial applications.



N. SEMENOV studied for three years at the Faculty of Mechanics and Mathematics of Lomonosov Moscow State University, then transferred to the Faculty of Mathematics at the Higher School of Economics, where I completed my bachelor's degree. I am currently not affiliated with any research institutions and am engaged in independent research.

APPENDIX A LLM PROMPTS USED TO BUILD MEMORY GRAPH BY MEMORIZE PIPELINE

Tables 7 and 8 presents LLM prompts employed in Memorize pipeline for extracting thesis and object memory triples, respectively, from unstructured natural language text. These prompts facilitate the transformation of textual data into a structured knowledge graph representation.

Type	Prompt
System	Objective: The main goal is to meticulously gather information from input [Text] and organize this data into a clear, structured knowledge graph. Requirements for building a knowledge graph: 1. Entities must depict concepts, similar to Wikipedia nodes. Entities must not include verbs and any other words which described motion, properties and etc. All the entities must be the real world objects or abstract concept. Use a structured thesises format to capture data. For example, from [Text] "Albert Einstein, born in Germany, is known for developing the theory of relativity" [Extracted thesises] should be "Albert Einstein was born in Germany ['Albert Einstein', 'Germany', 'birth']", "Albert Einstein developed Theory of Relativity ['Albert Einstein', 'developing', 'Theory of Relativity']". 2. Thesises MUST BE comprehension and consistent, so you can make them in form of sentense. You better make a much longer thesis than split it into two which inconsistent separately. For example, you better extract thesis "North exit from kitchen is blocked by door" than "kitchen has door" and "north exit is blocked by door" because without context of kitcher "north exit is blocked by door" can be related to every room at home. 3. You should extract only concrete knowledges, any assumptions must be described as hypothesis. For example, from [Text] "John have scored many points and potentially will be winner" [Extracted thesises] should be "John scored many points ['John', 'scoring', 'point']", "John could be winner ['John', 'winner']". and you should not extract "John will be winner ['John', 'winner']". 4. Remember that nodes must be an atomary units while thesis can be more complex and long. 5. Do not miss important information. If [Text] is "book involves story about knight, who needs to kill a dragon", thesises should be "book involves knight", "knight needs to kill dragon". 6. Several thesises can be extracted, that contain information about the same node. For example "kitchen contains apple" "kitchen contains table", "apple is on table". Do not miss this type of connections. 7. Do not use "none" as one of the entities. [Extracted thesises] should be returned in the following format: <pre>thesis_1 [list of entites for thesis_1]; thesis_2 [list of entites for thesis_2]; ... thesis_n [list of entites for thesis_n].</pre>
User	[Extracted thesises]:
Assistant	[Text]: {text}

TABLE 7. LLM prompts for extracting thesis memories (in the form of triples) from natural language text

Type	Prompt
System	Objective: The main goal is to meticulously gather information from the input [Text] and organize this data into a clear, structured knowledge graph. Knowledge graphs consists of a set of triplets. Each triplet contains two entities (subject and object) and one relation that connects these subject and object. The subject is the entity that takes or undergo the action expressed by the predicate. The object is the entity which is the factual object of the action. The information provided by each predicate can be summarized as a knowledge triplet of the form "subject predicate object". Requirements for building a knowledge graph: 1. Nodes must depict objects or concepts, usually in one or two words. Subjects and objects could be named entities or concepts describing a group of people, events, or abstract objects from the Wikidata knowledge graph. 2. Complex objects or concepts like "John position engineer in Google" must be splitted into simple ones like "John position engineer" and "John work at Google". 3. Length of the triplet should not be more than 7 words. 4. Extract only concrete knowledges, any assumptions must be described as hypothesis. For example, from [Text] "John have scored many points and potentially will be winner" extract "John scored many points", "John could be winner" and should not extract "John will be winner". 3. Object and subject in triplet must be an atomary units while relation can be more complex and long. 4. Do not miss important information. If [Text] is "book involves story about knight, who needs to kill a dragon", [Extracted Triplets] should be "book involves knight", "knight needs to kill dragon". 5. Several triplets can be extracted, that contain information about the same node. For example: "kitchen contains apple", "kitchen contains table", "apple is on table". Do not miss this type of connections. 6. Do not use "none" as objects/subject in triplet. [Extracted triplets] should be returned in the following format: subject_1 relation_1 object_1; subject_2 relation_2 object_2; ... subject_n relation_n object_n. Examples of [Text] and expected [Extracted Triplets] are presented in a list below: #### Example 1 [Text]: Albert Einstein, born in Germany, is known for developing the theory of relativity. [Extracted Triplets]: Albert Einstein country of birth Germany; Albert Einstein developed Theory of Relativity.
User	[Extracted Triplets]:
Assistant	[Text]: {text}

TABLE 8. LLM prompts for extracting object memories (in the form of triples) from natural language text

APPENDIX B LLM PROMPTS USED TO FIND OUTDATED INFORMATION IN CONSTRUCTED MEMORY GRAPH

Tables 9 and 10 displays LLM prompts employed in Memorize pipeline for identifying stale thesis-related and object memories, respectively, in memory graph.

Type	Prompt
System	You are a helpful assistant.
User	You will be provided with list of existing theses and list of new theses. The theses denote facts about the environment where the player moves. The player takes actions and the environment changes, so some theses from the list of existing theses can be replaced with one of the new theses. For example, the player took the item from the locker and the existing thesis "item is in locker" should be replaced with the new thesis "item is in inventory". Sometimes there are no theses to replace: Example of existing theses: ["Golden locker is open", "Room K is west of Room I", "Room K has exit to east"] Example of new theses: ["Room T is north of Room N", "Room T has exit to south"] Example of replacing: []. Nothing to replace here Sometimes several theses can be replaced with one: Example of existing theses: ["kitchen contains broom", "broom is on floor"] Example of new theses: ["broom is in inventory"] Example of replacing: ["broom is in inventory <- kitchen contains broom", "broom is in inventory <- broom is on floor"]. Because broom changed location from the floor in the kitchen to players inventory. Ensure that theses are only replaced if they contain redundant or conflicting information about the same aspect of an entity. Theses should not be replaced if they provide distinct or complementary information about entities compared to the new theses. Specifically, consider the relationships, properties, or contexts described by each thesis and verify that they align before replacement. If there is uncertainty about whether a thesis should be replaced, prioritize retaining the existing thesis over replacing it. When comparing existing and new theses, if they refer to different aspects or attributes of entities, do not replace them. Replacements should only occur when there is semantic duplication between an existing thesis and a new thesis. Example of existing theses: ["apple need to be cooked", 'knife used for cutting', 'apple has been sliced'] Example of new theses: ["apple is on table", 'kitchen contains knife', 'apple has been grilled'] Example of replacing: []. Nothing to replace here. These theses describe different properties of items, so they should not be replaced. Another example of when not to replace existing theses: Example of existing theses: ["brush is used for painting"] Example of new theses: ["brush is in art class"] Example of replacing: []. Nothing to replace here. These theses describe different properties of brush, so they should not be replaced. I repeat, do not replace theses if they carry different type of information about entities!!! It is better to leave a thesis, than to replace the one that has important information. Do not state that thesis needs to be replaced if you are not sure!! If you find thesis in Existing theses which semantically duplicate some thesis in New theses, replace such thesis from Existing theses. However do not replace thesis if they refer to different things. For every thesis you want to replace you must find replacement from new theses. If there is no clear replacement, you must not replace existing thesis. The replacements should contain information about the same properties as the statements being replaced, but must include more recent information. You MUST save existing thesis if it contains unique or actual information about entities. For example, you MUST NOT replace thesis "running is done with sneakers" with "sneakers located at store". #### Generate only list of replacing, no descriptions are needed. Existing theses: {ex_theses}. New theses: {new_theses}. #### Warning! Replacing must be generated strictly in following format: ["new_thesis_1 <- outdated_thesis_1"; "new_thesis_2 <- outdated_thesis_2"; ...], you MUST NOT include any descriptions in answer. Replacing:
Assistant	-

TABLE 9. LLM prompts for identifying outdated thesis memories

Type	Prompt
System	You are a helpful assistant.
User	[Extracted thesises]:You will be provided with list of existing triplets and list of new triplets. Triplets are in the following format: "subject, relation, object". The triplets denote facts about the environment where the player moves. The player takes actions and the environment changes, so some triplets from the list of existing triplets can be replaced with one of the new triplets. For example, the player took the item from the locker and the existing triplet "item, is in, locker" should be replaced with the new triplet "item, is in, inventory". Sometimes there are no triplets to replace: Example of existing triplets: "Golden locker, state, open"; "Room K, is west of, Room I"; "Room K, has exit, east". Example of new triplets: "Room T, is north of, Room N"; "Room T, has exit, south". Example of replacing: []. Nothing to replace here Sometimes several triplets can be replaced with one: Example of existing triplets: "kitchen, contains, broom"; "broom, is on, floor". Example of new triplets: "broom, is in, inventory". Example of replacing: [["kitchen, contains, broom" -> "broom, is in, inventory"], ["broom, is on, floor" -> "broom, is in, inventory"]]. Because broom changed location from the floor in the kitchen to players inventory. Ensure that triplets are only replaced if they contain redundant or conflicting information about the same aspect of an entity. Triplets should not be replaced if they provide distinct or complementary information about entities compared to the new triplets. Specifically, consider the relationships, properties, or contexts described by each triplet and verify that they align before replacement. If there is uncertainty about whether a triplet should be replaced, prioritize retaining the existing triplet over replacing it. When comparing existing and new triplets, if they refer to different aspects or attributes of entities, do not replace them. Replacements should only occur when there is semantic duplication between an existing triplet and a new triplet. Example of existing triplets: "apple, to be, cooked", 'knife, used for, cutting', 'apple, has been, sliced' Example of new triplets: "apple, is on, table", 'kitchen, contains, knife', 'apple, has been, grilled'. Example of replacing: []. Nothing to replace here. These triplets describe different properties of items, so they should not be replaced. Another example of when not to replace existing triplets: Example of existing triplets: "brush, used for, painting". Example of new triplets: "brush, is in, art class". Example of replacing: []. Nothing to replace here. These triplets describe different properties of brush, so they should not be replaced. I repeat, do not replace triplets, if they carry different type of information about entities!!! It is better to leave a triplet, than to replace the one that has important information. Do not state that triplet needs to be replaced if you are not sure!! If you find triplet in Existing triplets which semantically duplicate some triplet in New triplets, replace such triplet from Existing triplets. However do not replace triplets if they refer to different things. ### Generate only replacing, no descriptions are needed. Existing triplets: {ex_triplets}. New triplets: {new_triplets}. #### Warning! Replacing must be generated strictly in following format: [[outdated_triplet_1 -> actual_triplet_1], [outdated_triplet_2 -> actual_triplet_2], ...], you MUST NOT include any descriptions in answer. Replacing:
Assistant	-

TABLE 10. LLM prompts for detecting obsolete object memories

APPENDIX C LLM PROMPTS USED IN PROPOSED QA PIPELINE

Table 11 presents LLM prompts employed in QA pipeline at the second stage for extracting key entities from the original user question. Table 12 displays LLM prompts used in the fourth stage to generate contextually appropriate responses to user question.

Type	Prompt
System	You are an expert system that can extract [Key entities] from a given [Text]. [Key entities] is a noun or an object like persone, device, company and etc. Extracted entities must be presented in the following format: "<entitie1> <entitie2> ... <entitieN>", where <entitie...> is extracted [Key entities]. Your response only must contains extracted entities and do not include additional explanations of the obtained result. Examples of [Texts] and their corresponding extracted [Key entities] are listed below: #### Example 1 [Text]: Kayla has positive, negative or neutral opinion about video of Xiaomi 10Pro? [Key entites]: Kayla opinion video Xiaomi 10Pro #### Example 2 [Text]: Which device is better in battery life: Apple or k30u? [Key entites]: device battery life Apple k30u. #### Example 3 [Text]: The majority of speakers have positive, neutral or negative sentiment about screen of Samsung? [Key entites]: speakers sentiment screen Samsung #### Example 4 [Text]: Which people have positive opinion about video of Xiaomi 10Pro on 25.11.2020? [Key entites]: people opinion video Xiaomi 10Pro 25.11.2020
User	Extract [Key entites] from the given [Text]. [Text]: {text}
Assistant	[Key entites]:

TABLE 11. LLM prompts for extracting key entities from natural language text

Type	Prompt
System	You are a system that is trying to generate an [Answer] for a given [Question], based on provided [Contexts] in a list format. If there is no relevant information in a given [Contexts] for generating the suitable answer or the [Contexts] is "<Empty >", then you should generate the following: "< NotEnoughInfo >". Before generating the final answer you should present your [Chain of thoughts] that lead to the [Answer]. The format you must match for generating response is presented below: [Chain of thoughts]: <chain-of-thoughts> [Answer]: <final-answer> ,where <chain-of-thoughts> is your reasoning path based on given [Question] and [Contexts] that concludes to the [Answer] and <final-answer> is your final [Answer] to the [Question]. Examples of [Questions], [Contexts] and expected [Answers] are presented in a list below: #### Example 1 [Example Question]: Whose opinions from Anthony and Grace about devices are most similar to Faith's? [Example Contexts]: - Xiaomi (kind: device) opinion (person: Anthony; opinion: hard) maintenance point (kind: feature) - mate30pro (kind: device) opinion (person: Anthony; opinion: not as good as) signal (kind: feature) - iPhone (kind: device) opinion (person: Anthony; opinion: not as good as) signal (kind: feature) - Xiaomi 12 (kind: device) opinion (person: Grace; opinion: beat) charging speed (kind: feature) - Xiaomi (kind: device) opinion (person: Faith; opinion: problem) product control (kind: feature) - k30s (kind: device) opinion (person: Faith; opinion: not as good) film effect (kind: feature) - red rice (kind: device) opinion (person: Faith; opinion: not as good) film effect (kind: feature) [Example Chain of thoughts]: The task is to compare Anthony's and Grace's opinions to Faith's opinions about devices. Faith has two types of opinions: "problem" with the Xiaomi device and "not as good" with both k30s and red rice devices. We will look for similar expressions of dissatisfaction from Anthony and Grace. Grace's opinion about Xiaomi 12 is "beat", which is not similar to any of Faith's negative opinions. Anthony's opinions include "hard" for Xiaomi and "not as good as" for mate30pro and iPhone with respect to the signal feature. "Not as good as" matches Faith's "not as good". [Example Answer]: Anthony #### Example 2 [Example Question]: Whose opinions from Amanda and Arianna about manufacturers are most similar to Joshua's? [Example Contexts]: - Xiaomi 12 (kind: device) opinion (person: Grace; opinion: beat) charging speed (kind: feature) - Xiaomi (kind: device) opinion (person: Faith; opinion: problem) product control (kind: feature) - k30s (kind: device) opinion (person: Faith; opinion: not as good) film effect (kind: feature) [Example Chain of thoughts]: The task is to compare Amanda's and Arianna's opinions to Joshua's opinions about manufacturers. The provided contexts does not allow to determine which opinions from Amanda and Arianna about manufacturers are most similar to Joshua's. [Example Answer]: < NotEnoughInfo > #### Example 3 [Example Question]: The majority of speakers have positive, neutral or negative sentiment about signal of Apple? [Example Contexts]: < Empty > [Example Chain of thoughts]: The information for a give question is not provided. [Example Answer]: < NotEnoughInfo >
User	Generate [Answer] for the [Question] based on provided [Contexts]. [Question]: {q} [Contexts]: {c}
Assistant	[Chain of thoughts]:

TABLE 12. LLM prompts for conditional generation of answer to user question

APPENDIX D PREPROCESSING OPERATIONS FOR EVALUATION DATASETS

For the original HotpotQA dataset, the distractor/validation subset was selected, comprising 7405 question-answer (QA) pairs and 13781 unique contexts. QA pairs were then filtered to exclude those with associated contexts falling outside a specified length range (measured in characters), retaining only contexts between 64 and 1024 characters in length. This filtering process resulted in 13291 remaining contexts. Finally, first 2000 QA pairs and their corresponding contexts were extracted, yielding a final subset of 3933 unique contexts.

For the original TriviaQA dataset, the rc.wikipedia/validation subset was selected, comprising 7993 question-answer (QA) pairs. Given the extensive length of the contexts in this dataset, they were segmented into smaller fragments (chunks) using the "RecursiveCharacterTextSplitter" class from the LangChain library. The following hyperparameters were applied: a chunk size of 1024 characters, separators set to double newline characters ("\n\n"), a chunk overlap of 64 characters, the "len" function for length calculation, and is_separator_regex set to "False". This preprocessing yielded 278384 unique text fragments. Subsequently, QA pairs were excluded if their associated text fragments fell outside the specified length bounds (minimum 64 and maximum 1024 characters), resulting in 13291 retained fragments. Additionally, since the original contexts were split without explicit tracking of which fragment contained the necessary information to answer a given question, if any fragment from a context was discarded, all remaining fragments from that context were also removed to ensure coherence. This step further reduced the dataset to 9975 unique fragments. Finally, the first 500 QA pairs and their corresponding relevant fragments were selected, leaving a total of 4925 unique fragments for analysis.

Thus, evaluation sets for evaluation of proposed/implemented Memorize and QA pipelines were obtained. The characteristics of the obtained subsets of HotpotQA, TriviaQA and DiaSQ datasets can be found in Table 13.

Dataset	QA-pairs							Relevant contexts			
	Amount	Questions length (in characters)			Answers length (in characters)			Amount	Length (in characters)		
		median	mean	std	median	mean	std		median	mean	std
DiaSQ	5698	114	109.44	18.66	8	7.57	2.30	3483	556	613.00	324.35
HotpotQA	2000	87	92.98	32.62	13	15.29	11.87	3933	384	413.72	201.05
TriviaQA	500	66	76.37	39.46	9	10.17	5.76	4925	807	765.11	196.32

TABLE 13. Extended characteristics of datasets, used to evaluate proposed Memorize and QA pipelines

APPENDIX E RETRIEVAL HYPERPARAMETERS

- **A*:** h_metric_name – ip; max_depth – 10; max_passed_nodes – 150.
- **WaterCircles:** strict_filter – True; hyper_num – 15; episodic_num – 15; chain_triplets_num – 25; other_triplets_num – 6; do_text_pruning – False.
- **BeamSearch:** max_depth – 5; max_paths – 10; same_path_intersection_by_node – False; diff_paths_intersection_by_node – False; diff_paths_intersection_by_rel – False; mean_alpha – 0.75; final_sorting_mode – mixed.

APPENDIX F LLM-AS-A-JUDGE INSTRUCTIONS

To ensure the reproducibility of the obtained results, LLM inference was conducted using a deterministic generation strategy. The following hyperparameters were applied: num_predict – 2048, seed – 42, temperature – 0.0, and top_k – 1. The Qwen2.5 7B model, sourced from the Ollama repository, was prompted to evaluate whether the outputs of the proposed QA pipeline correctly answered the given questions. The specific LLM prompts used for this assessment are provided in Table 14.

Type	Prompt
System	You are an expert that evaluating the quality of the answer to the question. You are given: a [Question], a [Ground Truth Answer] that is definitely correct, and a [Generated Answer] that you must score as 0 or 1. A score of 1 means that the [Generated Answer] is correct, a score of 0 means that the [Generated Answer] is incorrect. You need to compare the [Generated Answer] with the [Ground Truth Answer], taking into account the [Question] itself. The [Ground Truth Answer] and the [Generated Answer] can be of different lengths and contain different amounts of information, so consider whether the answer is given to the interrogative word from the question: if in the [Question] asking "who", then the [Generated Answer] must name a person, character, or animal; if in the [Question] asking "how many", then the answer must contain a number. Examples of [Questions], [Ground Truth Answer], [Generated Answer] and its corresponding [Rate] are listed below: #### Example 1 [Question]: Where does Everybody rank on the Hot Dance Club Songs chart? [Ground Truth Answer]: Everybody climbs to number 3 on the Hot Dance Club Songs chart. [Generated Answer]: 3rd place. [Score]: 1 #### Example 2 [Question]: Which area has many populous villages? [Ground truth answer]: In the area of the lands of Colchis. [Generated answer]: There are many populous villages in this area. [Score]: 0 #### Example 3 [Question]: How many volumes is the History of Rome published in? [Ground Truth Answer]: The history of Rome is published in four volumes. [Generated Answer]: Answer: In three volumes. [Score]: 0 #### Example 4 [Question]: What kind of head did Michelangelo have? [Ground Truth Answer]: Michelangelo had a round head. [Generated Answer]: Michelangelo's head was round, his forehead was square, furrowed, and with pronounced brow ridges. [Score]: 1
User	[Question]: {q} [Ground Truth Answer]: {gold_answer} [Generated Answer]: {gen_answer}
Assistant	[Score]:

TABLE 14. LLM prompts used in LLM-as-a-Judge framework

APPENDIX G CHARACTERISTICS OF CONSTRUCTED MEMORY GRAPHS

To evaluate the QA pipeline we constructed 14 memory graphs based on the given dataset and LLM configurations. The structural characteristics of the generated graphs are detailed in Table 15.

Results indicate that certain graphs experienced parsing errors during the extraction of LLM responses, resulting in incomplete storage of contextual information. Across the evaluated datasets, the average parsing error rates were as follows: DiaASQ (7.0%), HotpotQA (6.3%), and TriviaQA (7.3%). For the DiaASQ dataset, DeepSeekR1 7B produced highest number of thesis and object vertices, as well as hyper-edges, while GPT4o-mini extracted largest number of unique object-typed edges. In contrast, for the HotpotQA and TriviaQA datasets, Qwen2.5 7B generated most thesis and object vertices, whereas DeepSeekR1 7B again yielded the highest number of hyper-edges. Based on the obtained results, we can model the structural characteristics of memory graphs generated by each evaluated system: see Figure 16.

The data reveal significant variation in LLM response parsing accuracy across models. Specifically, Qwen2.5 7B and

Llama3.1 8B demonstrate lowest error rates (0.02% each), while DeepSeek V3 exhibits the highest parsing error rate (31.21%). Intermediate performance is observed in DeepSeekR1 7B (0.29%) and GPT4o mini (9.87%). Regarding memory graph composition, DeepSeekR1 7B and Qwen2.5 7B yield the most comprehensive representations, generating the highest number of thesis/object memories and associated edges. Further analysis of vertices creation efficiency per contextual unit shows Qwen2.5 7B achieves superior granularity, producing the largest number of thesis/object memories while maintaining contextual coherence: see Table 17.

LLM	Number of contexts to store in graph	Nubmer of vertices			Number of edges			Average number of matching vertices (by type)			
		episodic	thesis	object	episodic	thesis	object	object neighbours (to episodic vertices)	thesis neighbours (to episodic vertices)	object neighbours (to thesis vertices)	object neighbours (to object vertices)
		DiaASQ									
DeepSeek R1 7B	3483	3477	34039	30097	138584	133301	34049	30.06	10.01	3.91	1.99
Qwen2.5 7B		3482	32512	28420	129974	111618	31290	27.98	9.48	3.43	2.04
Llama3.1 8B		3482	29655	20014	100045	77467	28063	20.21	8.52	2.61	2.25
GPT4o mini		3482	31361	28172	125403	96667	39477	27.00	9.00	3.08	1.89
DeepSeek V3		2270	21482	18416	80118	64349	28877	25.83	9.46	2.99	2.11
		HotpotQA									
DeepSeek R1 7B	3933	3921	27137	55254	121359	111174	33714	24.00	7.34	4.09	1.55
Qwen2.5 7B		3933	31795	56078	119387	106037	38460	22.26	8.15	3.33	1.36
Llama3.1 8B		3933	26364	40601	89506	75332	29021	16.04	6.79	2.85	1.47
GPT4o mini		3933	30777	48771	105524	93791	42599	18.99	7.94	3.04	1.36
DeepSeek V3		2713	20164	35291	73242	63921	34621	19.55	7.53	3.17	1.41
		TriviaQA									
DeepSeek R1 7B	4925	4905	48855	96132	213481	201861	52019	33.50	10.56	4.13	1.53
Qwen2.5 7B		4923	52835	109900	220991	188780	62043	34.15	10.87	3.57	1.27
Llama3.1 8B		4922	45241	72285	158202	127389	46757	22.83	9.35	2.81	1.54
DeepSeek V3		3506	37496	68602	143933	122480	61821	30.32	10.75	3.26	1.42

TABLE 15. Characteristics of constructed memory graphs for QA experiments

LLM	Number of contexts to store in graph	Number of vertices			Number of edges			Average number of adjacent vertices (by type)			
		episodic	thesis	object	episodic	thesis	object	object neighbours (to episodic vertices)	thesis neighbours (to episodic vertices)	object neighbours (to thesis vertices)	object neighbours (to object vertices)
DeepSeek R1 7B	4113	4101	36677	60494	157808	148778	39927	29.18	9.30	4.04	1.69
Qwen2.5 7B		4112	39047	64799	156784	135478	43931	28.13	9.5	3.44	1.55
Llama3.1 8B		4112	33753	44300	115917	93396	34613	19.69	8.22	2.75	1.75
GPT4o mini		3707	31069	38471	115463	95229	41038	22.99	8.47	3.06	1.625
DeepSeek V3		2829	26380	40769	99097	83583	41773	25.23	9.24	3.14	1.64

TABLE 16. Average/expected characteristics of knowledge graphs in case of using given LLM models for their construction

LLM	Vertex		Edge	
	thesis	object	thesis	object
DeepSeek R1 7B	9	15	36	10
Qwen2.5 7B	10	16	33	11
Llama3.1 8B	8	11	23	8
GPT4o mini	8	10	26	11
DeepSeek V3	9	14	30	15

TABLE 17. Number of unique vertices/edges that were added to the graph when processing (using a given LLM model) and storing one episodic memory (episodic vertex)

In addition to characteristics of constructed graphs, we collected information about time and speed of the Memorize pipeline, which responsible for parsing incoming unstructured texts in natural language and storing them in memory model: see Table 18.

LLM / Dataset	Memory construction time (hours) and speed (contexts per minute)					Mean:
	Qwen2.5 7B	DeepSeek R1 7B	Llama3.1 8B	GPT4o mini	DeepSeek V3	
DiaASQ	23.5 / 2.47	57.5 / 1.00	32 / 1.81	32 / 1.81	68 / 0.85	42.6 / 1.58
HotpotQA	61.5 / 1.06	47.5 / 1.38	27 / 2.42	34 / 1.92	72 / 0.91	48.4 / 1.53
TriviaQA	90 / 0.91	-	90 / 0.91	-	80 / 1.02	86.6 / 0.94
Mean:	58.3 / 1.48	52.5 / 1.19	<u>49.7 / 1.71</u>	<u>33 / 1.86</u>	73.3 / 0.92	

TABLE 18. Time and speed of memory graphs construction based on a given dataset and LLM, which were used in proposed QA pipeline evaluation

Table 18 shows that highest speed of parsing and saving information into memory is observed in configurations of Memorize pipeline there GPT4o mini is set as LLM (1.86); in second place is LLama3.1 8B, that was deployed in Ollama Docker container on local machine.

It is also important to note the amount of disk space that constructed memory is required. When using Milvus as a database for storing vector representations of text, on selected datasets, one memory graph in our implementation occupies, approximately, 80-90 GB. If Qdrant is used as a vector storage, the same graphs will occupy, approximately, 4-6 GB each.

APPENDIX H COMPARISON WITH EXISTING RAG AND GRAPH-RAG METHODS

Based on experimental results, a comparative table, summarizing best-performing QA configurations by Exact Match metric, was compiled (see Table 19).

LLM / Dataset	Qwen2.5 7B	DeepSeek R1 7B	Llama3.1 8B	GPT4o mini	DeepSeek V3
DiaASQ	0.22 / BS + WC / T	0.11 / AS / E	0.18 / AS / E	0.47 / BS + WC / E	0.46 / BS + WC / O
HotpotQA	0.18 / AS / all	0.14 / AS / E	0.37 / BS / O	0.59 / BS + WC / all	0.6 / BS + WC / T
TriviaQA	0.2 / BS / E	0.18 / AS / E	0.47 / BS / E	-	0.62 / BS + WC / all
Mean:	0.2	0.14	0.34	0.53	<u>0.56</u>

TABLE 19. Best QA configurations ranked by Exact Match metric across all experiments. The corresponding cells contains Exact Match score, used retrieval algorithm and the type of restriction applied to the memory graph during traversal. Shortcuts for retrieval algorithms: BA – BeamSearch; AS – A*; BS + AS – hybrid of BeamSearch and A*; BS + WC – hybrid of BeamSearch and WaterCircles. Shortcuts for graph restrictions: all – no restrictions applied; E – episodic vertices excluded from traversal; T – thesis vertices excluded; O – object vertices excluded.

As shown in Table 19, Qwen2.5 achieve best performance (0.2) among the 7B models. Among all evaluated configurations, highest overall effectiveness (0.56) was reached by setups incorporating DeepSeek V3. Notably, top-performing 7B configurations predominantly relied on A*, especially under constraint that traversal restricted through episodic vertices. In contrast, best DeepSeek V3 configurations frequently adopted a hybrid strategy combining BeamSearch and WaterCircles. Across high-performing configurations more broadly, BeamSearch consistently appeared as a key component within the retrieval pipeline.

To evaluate the performance of optimal configurations derived from our framework against existing Retrieval-Augmented Generation (RAG) and GraphRAG approaches, we conducted a systematic literature review. The search was performed across five academic search engines: SciSpace, Scite, PaperDigest, Consensus, and Elicit. Using each engine, we selected the first ten publications in the search results for the following queries: in case of RAG methods – "Retrieval-augmented generation methods based on pretrained language models" and "RAG methods in NLP"; in case of GraphRAG methods – "RAG on Knowledge Graphs", "Enhancing RAG-approach with Knowledge Graphs", "Graph RAG" and "RAG with integration of Large Language Models (LLMs) and Knowledge Graphs (KGs)". The search was restricted to publications up to 2018. Subsequently, we applied a three-stage filtering process: (1) duplicate entries were removed, and papers introducing novel evaluation datasets were excluded; (2) works focusing on domain-specific applications without broader methodological contributions were discarded; (3) only studies providing comprehensive methodological descriptions and employing standardized benchmarks were retained. This process yielded two final sets of nine articles each, covering RAG and GraphRAG techniques, respectively. A comparative analysis of these methods against our framework's optimal graph construction and retrieval configuration is presented in Table 20, with performance measured using ExactMatch metric.

Dataset	RAG method									Our method
	REALM [22]	DPR [3]	RAG [23]	ColBERT-QA [24]	FiD [25]	EMDR2 [26]	RETRO [27]	Atlas [28]	RePLUG [29]	
TriviaQA	53.9	56.8	55.8	70.1	67.6	71.4	62.1	<u>79.8</u>	77.3	62.0
	GraphRAG method									
	ToG [30]	RoG [31]	PMKGE [32]	GRAG [33]	GNN-RAG [34]	ToG2.0 [35]	DoG [36]	GCR [37]	PDA [38]	
HotpotQA	41.0	43.0	42.6	36.1	43.0	40.9	45.3	45.9	36.5	<u>60.0</u>

TABLE 20. Comparison of existing RAG and GraphRAG methods with our proposed method on ExactMatch metric. On TriviaQA dataset QA configuration with DeepSeek V3 and combination of BeamSearch and WaterCircles algorithms (without restrictions on graph traversal) were used. On HotpotQA dataset QA configuration with GPT4o-mini and the same algorithm and graph restrictions were used.

As shown in Table 20, our proposed method achieves a 14.1% improvement over existing GraphRAG approaches in a specific configuration. However, it underperforms compared to standard RAG methods by 17.8%. This discrepancy can be attributed to the fact that the RAG baselines evaluated in this study employed Reader and Retriever models that were specifically fine-tuned on the same dataset used for evaluation. As established in prior works, such in-domain fine-tuning typically yields optimal performance, and evaluation on out-of-domain datasets would be expected to result in significant degradation of measured metrics.

We also reproduce and evaluate the quality of HippoRAG method on DiaASQ and HotpotQA datasets: DiaASQ – 0.53 (LLM-as-a-Judge); HotpotQA – 60.2 (Exact Match). It can be seen that our method shows comparable results or outperforms HippoRAG.

...